

Linguaggi Formali e Compilatori - Prof. Breveglieri e Crespi Reghizzi - Prova scritta 01/07/2005 ¹

COGNOME e NOME:..... Matricola:.....

Crocettare:

- Laurea Specialistica ◦ Vecchio Ordinamento ◦ Altro (specificare)

AVVERTENZA: L'esame è diviso in 5 parti

1. Espr. reg. e aut. finiti
2. Grammatiche
3. Esercitazioni Flex Bison (fascicolo separato)
4. Grammatiche e analisi sintattica
5. Traduzione e semantica

1 Espressioni regolari e automi finiti 20%

1. Dato il ling. di alfabeto $\Sigma = \{a, b, c\}$

$$L = \Sigma^* ab \Sigma^* \cap \neg(\Sigma^* bc \Sigma^*) \cap a \Sigma^*$$

- (a) Si elenchino la (o le) frasi di lunghezza ≤ 3
- (b) Si costruisca, commentando il procedimento seguito, l'automa che riconosce L .
- (c) (facoltativo) Si calcoli l'espressione regolare di L con i soli operatori di concatenamento, unione, stella e croce.

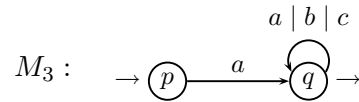
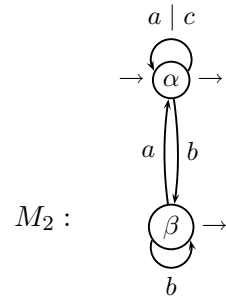
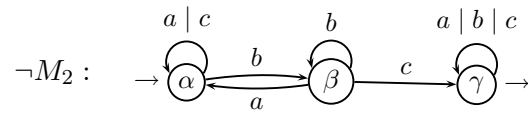
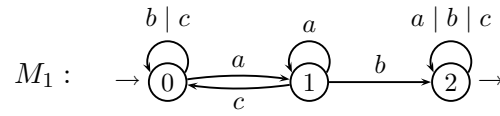
¹Tempo 3 ore (30 min. per parte 3). Libri e appunti personali possono essere consultati.

Soluzione

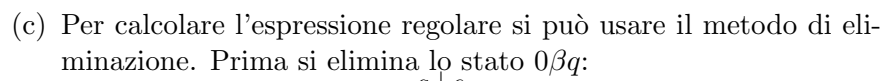
(a) ab, aab, aba, abb

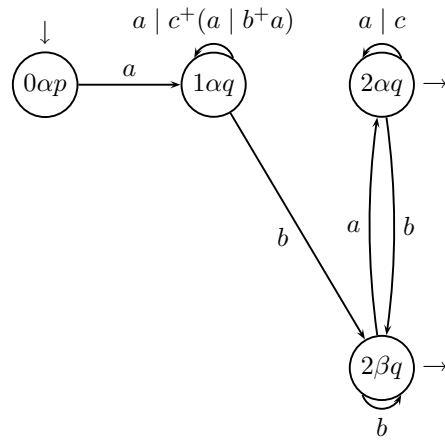
(b)

$$L = \underbrace{\Sigma^* ab \Sigma^*}_{M_1} \cap \underbrace{\neg(\Sigma^* bc \Sigma^*)}_{M_2} \cap \underbrace{a \Sigma^*}_{M_3}$$

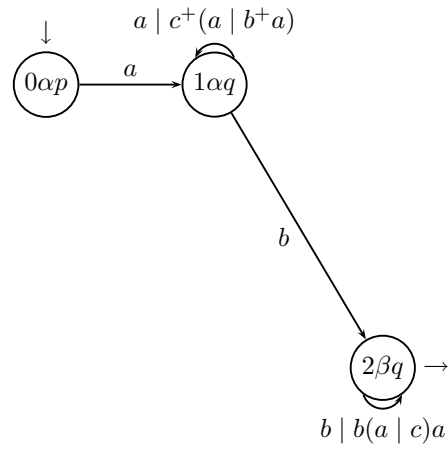


prodotto cartesiano $M_1 \times M_2 \times M_3 :$





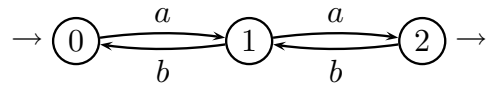
Eliminando lo stato $0\alpha q$:



Eliminando lo stato e $2\alpha q$:

$$L = a(a \mid c^+(a \mid b^+a))^*b(b \mid b(a \mid c)a)^*$$

2. È dato l'automa M :



- (a) Si consideri l'insieme dei suffissi propri delle frasi di $L(M)$ ossia il ling.

$$L_S = \{y \mid zy \in L(M) \wedge y \neq \varepsilon \wedge z \neq \varepsilon\}$$

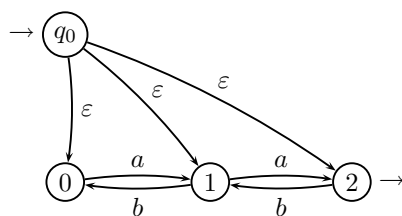
Si elenchino le frasi di lunghezza ≤ 2 del ling. L_S

- (b) Si costruisca il riconoscitore deterministico minimo di L_S spiegando il procedimento seguito.

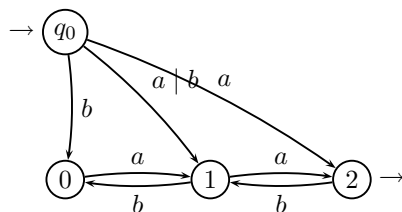
Soluzione

- (a) Frasi di lunghezza ≤ 2 : a, aa, ba
- (b) Per costruire il riconoscitore deterministico si vede che i suffissi propri sono:
- le stringhe non vuote che M riconosce partendo da uno stato non iniziale, cioè da 1 e da 2;
 - ogni frase $x \in L(M)$ è suffisso di un'altra frase $abx \in L(M)$.

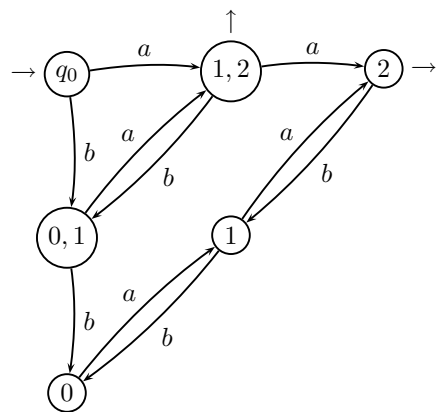
Si modifica allora l'automa in modo che tutti gli stati siano iniziali e, poiché ε non deve essere riconosciuta, nessuno stato iniziale sia finale:



Si eliminano le mosse spontanee:



Si determinizza:



2 Grammatiche 20%

1. Si consideri il ling. di Dyck di alfabeto $\text{'('}, \text{'('}, \text{'['}, \text{'['}$.
 - (a) Trovare la grammatica che genera tutte e sole le stringhe di Dyck dove: (1) se il numero totale di coppie di parentesi tonde è pari, il numero totale di coppie di parentesi quadre è dispari; mentre (2) se il numero totale di coppie di parentesi tonde è dispari, non si dice nulla rispetto alle parentesi quadre.
Esempi: $([]), (([])), ((([]([])))$.
Controesempi: $[], ((([]([])))$.

Soluzione

$$\begin{aligned} S &\rightarrow S_{pd} \mid S_{dp} \mid S_{dd} \\ S_{pp} &\rightarrow (S_{pp})S_{dp} \mid (S_{pd})S_{dd} \mid (S_{dp})S_{pp} \mid (S_{dd})S_{pd} \\ S_{pp} &\rightarrow [S_{pp}]S_{pd} \mid [S_{dp}]S_{dd} \mid [S_{pd}]S_{pp} \mid [S_{dd}]S_{dp} \\ &\dots \end{aligned}$$

2. Costruire la grammatica EBNF del linguaggio della teoria elementare delle classi, così definito. Sono ammesse classi di elementi e classi di classi, ecc, a qualsiasi livello di gerarchia e non necessariamente omogenee. Il linguaggio consta delle parti seguenti:

- (a) I simboli ‘,’ , ‘{’, ‘}’, ‘|’, ‘∪’, ‘∩’, ‘¬’, ‘⊂’, ‘∈’, ‘:=’ sono terminali.
- (b) Le lettere minuscole $a, \dots, z$ sono simboli terminali rappresentanti elementi.
- (c) Le lettere maiuscole $A, \dots, Z$ sono simboli terminali rappresentanti nomi di classi, di qualunque tipo.
- (d) Se C_1, C_2 sono classi ed e è un elemento, $C_1 \subset C_2$, $e \in C_1$ e $C_1 \in C_2$ sono predicati.
- (e) Se C_1, C_2 sono classi, $C_1 \cup C_2$, $C_1 \cap C_2$ e $\neg C_1$ sono classi.
- (f) Se A è un nome di classe ed e è un elemento, si possono formare classi secondo le regole seguenti:
 - i. $\{ \}$ (è la classe vuota)
 - ii. $\{e\}$
 - iii. $\{A\}$
 - iv. $\{e \mid \text{predicato}\}$
 - v. $\{A \mid \text{predicato}\}$
- (g) Se A è un nome di classe, la scrittura “ $A := \text{classe};$ ” è un assegnamento che definisce la nuova classe di nome A , con contenuto “classe” (una qualsiasi stringa di terminali e nonterminali che definisca una classe secondo le regole da (a-f)).

Il linguaggio consiste in una successione (non vuota) di assegnamenti a nomi di classi. La grammatica che lo genera deve essere non ambigua, e le precedenze sono $\neg$ precede $\cap$ precede $\cup$.

Esempi:

$A := \{e\}; \quad A := \{e\} \cup \{\{d\}\}; \quad A := \{e \mid e \in A\}; \quad A := B \cup (C \cap D);$
 $A := \{B \mid B \in \{e\} \cup D\}; \quad A := \{B \mid B \in \{C \mid C \subset D\}\};$

Quali aspetti semantici non sono modellabili mediante la grammatica proposta?

Soluzione

$$\begin{aligned} S &\rightarrow (\langle ass \rangle ;)^+ \\ \langle nomeclasse \rangle &\rightarrow ['A' - 'Z'] \\ \langle elem \rangle &\rightarrow ['a' - 'z'] \\ \langle ass \rangle &\rightarrow \langle nomeclasse \rangle := \langle espr \rangle \\ \langle espr \rangle &\rightarrow \langle term \rangle (\cup \langle term \rangle)^* \\ \langle term \rangle &\rightarrow \langle fatt \rangle (\cap \langle fatt \rangle)^* \\ \langle fatt \rangle &\rightarrow \neg \langle var \rangle \mid \langle var \rangle \\ \langle var \rangle &\rightarrow \{ \} \mid \{ \langle elem \rangle (\varepsilon \mid \langle pred \rangle) \} \mid \{ \langle nomeclasse \rangle (\varepsilon \mid \langle pred \rangle) \} \mid (\langle espr \rangle) \\ \langle pred \rangle &\rightarrow \langle espr \rangle \subset \langle espr \rangle \mid \langle elem \rangle \in \langle espr \rangle \mid \langle espr \rangle \in \langle espr \rangle \end{aligned}$$

3 Domanda relativa alle esercitazioni

Vedi fascicolo separato.

4 Grammatiche e analisi sintattica 20%

1. Calcolare gli insiemi guida delle grammatica data per $k = 1$, indicare dove essa non risulta $LL(1)$, e trovare il valore di $k > 1$ per cui la grammatica risulta $LL(k)$, motivando in breve la risposta.

Regola	Insieme Guida
$S \rightarrow XaBSc$	
$X \rightarrow aY$	
$X \rightarrow \varepsilon$	
$Y \rightarrow bX$	
$Y \rightarrow \varepsilon$	
$B \rightarrow abD$	
$D \rightarrow bD$	
$D \rightarrow b$	

2. Costruire il riconoscitore deterministico dei prefissi ascendenti e indicare in quale(i) stato(i) sono violate le condizioni LR(1).

$$S \rightarrow aSX \quad X \rightarrow bX \quad X \rightarrow b \quad X \rightarrow cX$$

3. Domanda facoltativa. Si consideri il linguaggio delle espressioni aritmetiche con tre operatori binari $+$, $-$ e $\times$, e un nome di variabile v . I due operatori $+$ e $\times$ sono di tipo prefisso, mentre l'operatore $-$ è di tipo postfisso (si tratta dunque di una forma mista prefissa-postfissa).

Al fine di interpretare la forma mista in modo univoco, si rifletta se sia necessario o no avere parentesi, e si dica eventualmente per quale(i) operatore(i). Qualunque sia la conclusione si scriva la grammatica che genera il linguaggio, in forma ambigua o no, a scelta, ma specificando quale delle due.

Suggerimento: si rifletta sulla stringa $v + \times v v \times v v - v -$.

Soluzione

$$((v (+ (\times v v) (\times v v)) -) v -) \\ (v (+ (\times v v) (\times (v v -) v)) -)$$

Mettere parentesi a $-$.

Grammatica facile ...

5 Traduzione e semantica 20%

1. Il ling. sorgente L_1 è una variante del ling. di Dyck, di alfabeto $\Sigma = \{('',''), b\}$, con b che designa uno spazio bianco. Le lettere b possono comparire in ogni posizione, anche ripetutamente.
 - (a) Si progetti una gramm. di traduzione ossia uno schema di traduzione (semplice, senza attributi semantici) per convertire una frase di L_1 nella corrispondente stringa in cui due parentesi sono sempre separate da una, e una sola, b e non ci sono altre b oltre a queste. Es.:

frase sorgente	frase pozzo
$bb(())$	$(b(b)b)$
(b)	(b)
bbb	ε
$()(bb(bbb)bb)b$	$(b)b(b(b)b)$

- (b) Si verifichi se è possibile costruire un traduttore deterministico partendo dalla vostra grammatica di traduzione.

Soluzione

Si presentano due soluzioni.

(a) Prima soluzione

- i. Gramm. di traduzione: l'idea per la gramm. sorgente è di modificare una grammatica di Dyck inserendo il nonterminale B che genera 0 o più b tra ogni coppia di parentesi. Nella gramm. pozzo il nonterminale B genererà invece una sola b . Perfezionando l'idea, si vede però che

- le stringhe b^* poste a prefisso e a suffisso non vanno tradotte in una lettera b ma cancellate. Occorre quindi differenziare tale caso dal precedente, introducendo un altro nonterminale A .
- è meglio che la frase b^* sia derivata direttamente dall'assioma.

gramm. sorgente	gramm. pozzo
$S \rightarrow AXA$	$S \rightarrow AXA$
$S \rightarrow A$	$S \rightarrow A$
$A \rightarrow bA$	$A \rightarrow A$
$A \rightarrow \varepsilon$	$A \rightarrow \varepsilon$
$X \rightarrow (BXB)BX$	$X \rightarrow (BXB)BX$
$X \rightarrow (B)BX$	$X \rightarrow (B)BX$
$X \rightarrow (BXB)$	$X \rightarrow (BXB)$
$B \rightarrow bB$	$B \rightarrow B$
$B \rightarrow \varepsilon$	$B \rightarrow b$

- ii. Per vedere se sia possibile costruire un traduttore deterministico, si può verificare se la gramm. sorgente sia LL(k) o possa essere trasformata in una gramm. equivalente LL(k). I nonterminali S e X non sono LL(1) ma fattorizzando a sin. le alternative conflittuali si ottiene la gramm. equivalente

gramm. sorgente	insieme guida
$S \rightarrow AW$	
$W \rightarrow XA$	(
$W \rightarrow \varepsilon$	$\neg$
$A \rightarrow bA$	b
$A \rightarrow \varepsilon$	$\neg, ($
$X \rightarrow (BZY$	
$Z \rightarrow XB$	(
$Z \rightarrow)$	)
$Y \rightarrow BX$	$b, ($
$Y \rightarrow \varepsilon$	$b, ($
$B \rightarrow bB$	b
$B \rightarrow \varepsilon$	(

nella quale resta però ancora un conflitto per Y . Questa tecnica non permette dunque di ottenere un traduttore a pila deterministico.

Resterebbe aperta la via di passare alla tecnica LR(1), modificando la grammatica di traduzione in modo che gli elementi dell'alfabeto possano trovarsi soltanto a suffisso.

(b) Seconda soluzione ***da Luca, circa:

$$\begin{aligned}
S_0 &\rightarrow B(BSB)B \parallel S_0 \rightarrow B(bBSB)B \\
S &\rightarrow B(BSB)BSB \parallel S \rightarrow B(bBSB)bBSB \\
S &\rightarrow B \parallel S \rightarrow B \\
B &\rightarrow b^* \parallel B \rightarrow \varepsilon
\end{aligned}$$

2. È data la sintassi di certi programmi contenenti istruzioni di assegnamento:

$$S \rightarrow AS \mid A$$

$$A \rightarrow i := E$$

$$E \rightarrow i + E \mid i \mid c$$

dove i è un identificatore di variabile, e c una costante.

- (a) Progettare una gramm. ad attributi per controllare che ogni variabile usata in un'espressione abbia un valore precedentemente calcolato da un assegnamento. Si veda l'albero dell'esempio. Si usino i seguenti attributi, aggiungendone altri se necessario:

predicato semantico: α of S , sintetizzato: dice se il programma supera il controllo.

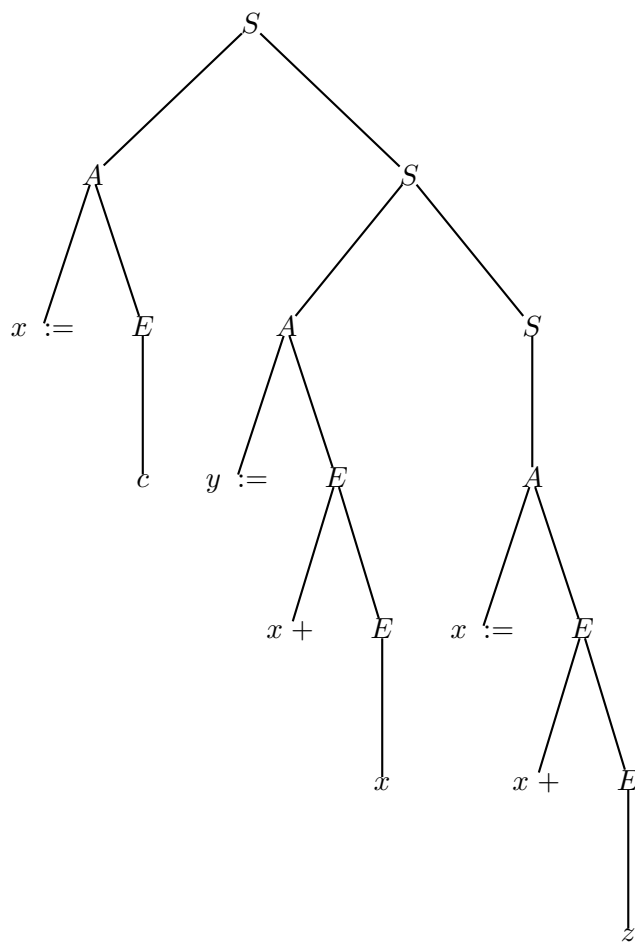
variabili usate: use of E , sintetizzato: l'insieme degli identificatori che compaiono nella espressione E .

variabili definite: def of A , ereditato: l'insieme degli identificatori inizializzati da un precedente assegnamento

Si scrivano le funzioni semantiche.

- (b) Disegnare sull'albero i valori degli attributi e le frecce delle dipendenze funzionali tra di essi.
- (c) Scrivere lo pseudocodice del valutatore semantico, supponendo che l'albero sintattico sia stato già costruito dal parsificatore.
- (d) Facoltativo: estendere i controlli semantici in modo da verificare se un assegnamento, diverso dall'ultimo, è inutile, ossia se il valore della variabile da esso calcolata non è utilizzato in un'espressione successiva.

Sintassi	Funzioni semantiche
$S \rightarrow AS$	
$S \rightarrow A$	
$A \rightarrow i := E$	
$E \rightarrow i + E$	
$E \rightarrow i$	
$E \rightarrow c$	



Il primo assegnamento è corretto, il secondo è corretto (ma è inutile), il terzo è scorretto poiché la variabile z non ha un valore.